Raahat राहत

Disaster photo reports, triaged by Cloudinary AI, delivered to responders on any network.
Hacktober HackDays 2026 (SheBuilds Delhi × MLH) · Tech for Social Good

In one line: Raahat ("relief") is a web app that takes disaster photos from ordinary people, uses Cloudinary AI to work out which are most urgent, and gives rescue teams a sorted, privacy-safe list on a map.

1. The problem

In a flood, earthquake or cyclone, people send photos to control rooms through WhatsApp groups and helplines. The control room ends up with hundreds of photos and these problems:

1. **No priority.** A family stuck on a roof sits next to a photo of a puddle. Someone has to look at every one.
2. **Duplicates.** The same photo is forwarded from 20 groups and looks like 20 emergencies.
3. **Fake or old photos.** Photos from past years or screenshots get shared as new. A 2017 Bihar flood photo circulated as 2025 Punjab flood footage.
4. **No location.** "Help, near the school" is not enough to send a boat.
5. **Privacy.** Photos of injured people and children spread publicly with faces visible.
6. **Bad network.** Control rooms in disaster areas often have very weak internet, so large photos won't load.

All of this wastes time, and in a rescue, time decides who survives. Floods in India's northeast affected more than 1.4 million people in June 2025 alone, and a study of the 2018 Kerala floods found social-media misinformation halted relief operations in some places.

2. Who uses it

Person	What they do in Raahat
Citizen	Opens a link, takes a photo, sends it with location, in Hindi or English.
Control-room coordinator	Sees all reports sorted by urgency on a map and dispatches teams.
Public, volunteers, media	See a safe public map: faces blurred, approximate locations only.

3. How it works, step by step

1. **Citizen reports.** On `/report` they take a photo; the app picks up GPS (or they tap the map). Optional note, e.g. "water up to the waist, 2 families on the roof". The photo is shrunk *on the phone* so it uploads on weak signal.
2. **Photo goes to Cloudinary**, where it is stored.

3. **AI reads the photo.** Cloudinary AI Vision answers:
   - Which of 10 hazard types is this?
   - How many people are visible? Is anyone in danger?
   - How deep is the water: ankle, knee, waist, chest?
   - Can a rescue vehicle reach the spot?
   - Is it a real photo, or a screenshot, meme or drawing?
   - A one-line summary in English and Hindi.
4. **Duplicate check.** Each photo gets a fingerprint (perceptual hash). The same photo sent again nearby is **merged**. A different photo of the same incident (same type, within 300 m, within 12 h) **confirms** it and raises its priority.
5. **Scoring.** Each report gets a 0–100 score and a level: **critical** **high** **moderate** **low**.
6. **Privacy.** Every face is pixelated in public images; public locations are rounded to about 110 m; the reporter's note is hidden from the public.
7. **Responders act** in the control room (`/respond`, passcode): sorted list and map, sharpened photo with an optional face reveal, 2G mode with few-KB thumbnails, a one-click WhatsApp alert card, and status buttons (verified, dispatched, resolved, rejected as false).

The 10 hazard types

People trapped or injured · Fire or heavy smoke · Building or bridge collapse · Landslide · Live wire or fallen pole · Flood · Road blocked · Relief needed (food, water, medicine) · Waterlogging · No emergency.

Not only floods: it works for earthquakes, cyclones, landslides, fires and urban waterlogging. Floods are the lead example because that is where photo overload is worst.

4. The priority score

Factor	Points
Type of danger (base)	Trapped 40 · Fire 35 · Collapse / Landslide 30 · Live wire 25 · Flood 20 · Road blocked / Relief 15 · Waterlogging 10 · None 0
Other hazards in the same photo	+25% of their base points
Someone in danger	+30
Each person visible (max 10)	+1.5 each
Water depth	Ankle +2 · Knee +8 · Waist +15 · Chest or deeper +22
Vehicles can't reach	+5
Each other independent report of the same incident (max 5)	+4 each
Looks fake or unrelated	Score × 0.3 and sent to human review (never deleted)

Levels: **critical** 70+ · **high** 45+ · **moderate** 25+ · **low** below 25. Score is capped at 100.

Because it is a simple visible formula, a coordinator can always see **why** one report is above another.

5. Tech stack

Part	Tool	Why
Website and server	Next.js 15 + TypeScript	Pages and backend in one project; free hosting on Vercel
Photo upload	Cloudinary Upload Widget (next-cloudinary)	Camera support; shrinks photo on the phone
AI	Cloudinary AI Vision (Analyze API)	Reads the photo and answers our questions
Image processing	Cloudinary transformations	Face blur, smart crop, compression, enhancement, alert card
Database	Cloudinary metadata + Search API	Report data lives on each photo; no separate database
Map	Leaflet + OpenStreetMap	Free, no API key
Hosting	Vercel	Free, gives a live link
Tests	Node test runner	16 tests: scoring, duplicates, AI-output parsing

Architecture

```
Phone  (Upload Widget: camera, shrink on device, signed upload)
   |
   v
Cloudinary asset  (tag: raahat_report)
   |
   v   POST /api/reports
Triage pipeline
   +- perceptual hash ............. duplicate detection
   +- AI Vision tagging ........... 10 hazard types
   +- AI Vision general ........... people, danger, depth, real/fake, EN+HI summary
   +- score + duplicate matching .. 0-100, severity, merge / confirm
   +- save as metadata + tags ..... Cloudinary is the database
   |
   v
Search API  -->  Control room (/respond)  +  Public map (/map)
   thumbnails: face pixelation, smart crop, tiny for 2G
   detail:     auto-enhanced photo
   alert card: text overlays -> WhatsApp-ready image
```

6. Cloudinary features used

Feature	What it does in Raahat
Upload Widget (signed)	Safe citizen uploads; photo shrinks on the phone first
AI Vision tagging	Sorts the photo into our 10 disaster types
AI Vision general	Counts people, judges water depth, flags fakes, writes Hindi summary
Perceptual hash (phash)	Spots forwarded copies of the same photo
`e_pixelate_faces`	Blurs every face automatically

`g_auto` smart crop	Thumbnails focus on the hazard, not the sky
`f_auto`, `q_auto`	Tiny thumbnails for 2G networks
`e_improve`, `e_sharpen`	Clearer night and rain photos
Video frame + `e_preview`	Video reports analysed too; short preview for responders
Text overlays (`l_text`)	WhatsApp alert card built from one URL
Contextual metadata, tags, Search API	The whole database

> "*Cloudinary isn't just where we store images. It's our upload system, our AI, our privacy layer, our image generator and our database.*"

7. What makes it stand out

1. **Real, serious problem**: saving minutes in a disaster saves lives.
2. **Built for India**: Hindi and English, weak networks, WhatsApp-style forwarding.
3. **Privacy by default**, which most projects ignore.
4. **Honest AI**: it ranks but never deletes; people check anything doubtful.
5. **Deep Cloudinary use**: many features, each with a clear reason.
6. **Works without the AI**: if the AI Vision add-on is unavailable, it reads keywords from the reporter's note and labels those reports "rule-based".

8. Honest limits

- AI can make mistakes, so people stay in control.
- One shared control-room passcode; a real version needs individual logins and an audit log.
- Citizens must open a website; next step is sending photos to a WhatsApp number.
- Not yet tested with a real control room.

9. Likely judge questions

What if the AI is wrong? The score is a simple visible formula, nothing is deleted, and doubtful reports go to human review.

How do you stop fake photos? AI flags screenshots and unrelated images; duplicate detection catches forwards. Detecting old photos from past disasters is on the roadmap.

Why no database? Cloudinary stores report data on each photo: one system to run and secure, and photo and data never go out of sync.

Who would use it? District disaster control rooms and NGOs that run volunteer WhatsApp groups during the monsoon.

What's next? WhatsApp intake, more Indian languages, offline sending, individual responder logins, old-photo detection.

10. Pitch in 3 minutes

Time	Say / show
0:00–0:30	Problem: control rooms drowning in unsorted photos; Kerala 2018 misinformation halted relief.
0:30–2:00	Live demo: report a photo in Hindi → forward the same photo and show it merge → control room → 2G mode → alert card.
2:00–2:30	How Cloudinary powers every layer.
2:30–3:00	Impact and next steps: WhatsApp intake, more languages, pilot with a district control room.

Sources: ReliefWeb / ECHO Daily Flash, 6 June 2025 (NE India floods); Vasudevan & Alathur, "Misinformation in Social Media During Disasters: A Case Study of the Flood in Kerala, India in 2018", IGI Global, 2022; Factly fact-check on the 2017 Bihar flood photo; Cloudinary AI Vision documentation.